

UNIVERSIDADE TECNOLÓGICA - POSTECH

Pós-Graduação em Software Engineering

TECH CHALLENGE - FASE 3

Automação de Infraestrutura, CI/CD com DevSecOps e GitOps

Disciplinas: Infraestrutura como Código · Pipelines de CI/CD · DevSecOps · GitOps

Integrantes do Grupo

#	Nome Completo	RM
1	Yuri José do Carmo	rm370037
2	Diego Felipe Rocha Silva	rm369588
3	Jhousyfran Muniz Costa	rm369476
4	Erick Saraiva de Sousa	rm369969
5	Regis Teruo Nomi	rm369601

Links de Entrega

Item	Link
Repositório GitHub	https://github.com/yurijcarmo/tech-challenge-3

Item	Link
Vídeo de Demonstração	(a inserir)
ArgoCD (interface web)	(a inserir - URL definida pela variável <code>argocd_domain</code> no <code>terraform.tfvars</code> , criada após <code>terraform apply</code>)

Acesso ao ArgoCD: usuário `admin`. A senha inicial é gerada pelo ArgoCD e obtida com:
`kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d`

Índice

1. [Resumo Executivo](#)
2. [Problema e Contexto](#)
3. [Arquitetura Geral da Solução](#)
4. [Infraestrutura como Código - Terraform](#)
5. [Pipeline de CI/CD com DevSecOps - GitHub Actions](#)
6. [Entrega Contínua e GitOps - ArgoCD](#)
7. [Decisões Técnicas Justificadas](#)
8. [Segurança do Repositório](#)
9. [Desafios e Limitações](#)
10. [Estimativa de Custos AWS](#)

1. Resumo Executivo

Este relatório documenta o trabalho desenvolvido na Fase 3 do Tech Challenge, cujo objetivo foi automatizar completamente a infraestrutura e o ciclo de vida dos 5 microsserviços do sistema **ToggleMaster** - plataforma de gerenciamento de feature flags.

Implementamos as três práticas exigidas pelo enunciado:

- **Infraestrutura como Código (IaC)** com Terraform, organizando toda a infraestrutura AWS em módulos independentes: rede, cluster Kubernetes, bancos de dados, filas, repositórios de imagem e componentes do cluster. O estado da infraestrutura é gerenciado de forma centralizada em um bucket S3 com mecanismo de locking.
- **CI/CD com DevSecOps** via GitHub Actions, com pipelines reutilizáveis que cobrem compilação, testes unitários, análise de qualidade de código, varredura de

vulnerabilidades em dependências (SCA), análise estática de segurança no código fonte (SAST), construção da imagem Docker, varredura da imagem, publicação no ECR e atualização automática dos manifestos GitOps. Vulnerabilidades críticas bloqueiam o pipeline com `exit-code: 1`.

- **GitOps com ArgoCD**, onde o repositório Git é a única fonte de verdade sobre o estado do ambiente. O ArgoCD monitora continuamente o repositório e sincroniza o cluster automaticamente a cada mudança detectada.

O trabalho foi conduzido com restrições do ambiente AWS Academy, que impede a criação de roles IAM. Adotamos como estratégia manter o código original de conta pessoal comentado nos arquivos, com a alternativa para Academy ativa - preservando a demonstração do conhecimento técnico completo.

2. Problema e Contexto

O enunciado apresentou um cenário de sistema em produção com os seguintes problemas estruturais:

Problema	Impacto	Nossa Solução
Infraestrutura criada manualmente no console	Impossível reproduzir exatamente. Um ambiente perdido significa recriar clicando	Terraform: toda infraestrutura em código versionado
Deploys manuais com <code>kubectl apply</code>	Conflitos de versão, ausência de auditoria, risco de erro humano	GitOps: o cluster só muda por commits no repositório
Credenciais hardcoded em arquivos	Risco crítico de vazamento no repositório	<code>.gitignore</code> , ESO, secrets GitHub, <code>.env.example</code>
Vulnerabilidades chegando à produção	Impacto em segurança sem detecção prévia	SCA + SAST + scan de imagem no pipeline
Sem testes automatizados no deploy	Regressões chegam aos usuários	Build + testes unitários antes de qualquer deploy

3. Arquitetura Geral da Solução



4. Infraestrutura como Código - Terraform

4.1 Organização em Módulos

Optamos por organizar a infraestrutura em módulos independentes dentro de `infra/modules/`. Cada módulo encapsula um conjunto coeso de recursos AWS e expõe apenas as entradas (`variables.tf`) e saídas (`outputs.tf`) necessárias para os outros módulos.

```
infra/
├── backend.tf           ← backend remoto S3
├── modules.tf           ← composição dos módulos
├── variables.tf         ← definição de variáveis
├── terraform.tfvars.example ← template de configuração (commitado)
├── modules/
│   ├── network/        ← VPC, subnets, IGW, NAT, route tables
│   ├── cluster/        ← EKS control plane + IAM
│   ├── managed-node-group/ ← EC2 worker nodes + IAM
│   ├── rds/            ← PostgreSQL
│   ├── elasticache/    ← Redis
│   ├── dynamodb/      ← tabela NoSQL
│   ├── sqs/            ← fila de mensagens
│   ├── ecr/            ← repositórios de imagem Docker
│   ├── addons-eks/     ← ArgoCD, NGINX, ExternalDNS, ESO, KEDA
│   ├── apps/           ← ArgoCD Applications para os 5 microsserviços
│   └── aws-loadbalacer-controller/ ← AWS LBC + IAM
```

Justificativa para modularização: a separação em módulos permite reutilização em outros projetos, facilita testes isolados e melhora a rastreabilidade de mudanças - uma alteração no módulo de RDS não afeta o código do módulo de rede. Para um projeto com múltiplos integrantes, isso também reduz conflitos de merge no Git, pois cada integrante trabalha em módulos distintos.

4.2 Rede - `modules/network/`

Recurso	Configuração	Justificativa
VPC	CIDR 10.0.0.0/16	Espaço de endereçamento privado amplo para crescimento
Subnets públicas	2 (zonas 1a e 1b)	Alta disponibilidade - 2 zonas distintas
Subnets privadas	2 (zonas 1a e 1b)	Nodes e bancos sem IP público exposto
Internet Gateway	1	Saída à internet das subnets públicas
NAT Gateway	2 (1 por zona)	Nodes privados acessam internet sem IP público
Route Tables	Separadas por tipo	Roteamento diferenciado público x privado

Decisão: nodes nas subnets privadas. Adotamos o princípio de menor exposição: servidores EC2 que rodam os containers não devem ter IP público. O tráfego de entrada dos usuários passa pelo Load Balancer (subnets públicas) e é roteado internamente para os pods. Isso reduz a superfície de ataque.

Decisão: 2 zonas de disponibilidade. Para o cenário de produção implícito no enunciado, uma única zona criaria ponto único de falha. Com 2 zonas, uma falha na zona 1a não interrompe o serviço - os pods na zona 1b continuam atendendo.

4.3 Cluster EKS - `modules/cluster/`

O cluster EKS (Elastic Kubernetes Service) é o ambiente gerenciado de Kubernetes onde os 5 microsserviços do ToggleMaster rodam. A AWS gerencia o plano de controle - API server, etcd, scheduler - e provisionamos os worker nodes via `managed-node-group`.

```
resource "aws_eks_cluster" "eks_cluster" {
  name     = "${var.prefix}-${var.project}-cluster"
  role_arn = data.aws_iam_role.lab_role.arn

  vpc_config {
    subnet_ids         = [var.public_subnet_1a_id, var.public_subnet_1b_id]
    endpoint_private_access = true
    endpoint_public_access = true
  }
}
```

`endpoint_private_access = true`: comunicação interna entre nodes e API server ocorre pela rede privada da VPC, sem sair para a internet - mais seguro e mais rápido.

4.4 Node Group - `modules/managed-node-group/`

Os worker nodes foram configurados como instâncias `t3.medium` com scaling fixo de 2 réplicas. Escolhemos `t3.medium` por equilibrar custo e capacidade para workloads de desenvolvimento: 2 vCPU e 4 GB de RAM são suficientes para os 5 serviços com suas réplicas.

Os nodes rodam exclusivamente nas subnets privadas, sem IP público. O acesso ao ECR para download de imagens ocorre via NAT Gateway.

4.5 Bancos de Dados, Cache e Mensageria

Módulo	Recurso	Tipo	Finalidade
<code>rds/</code>	3 instâncias PostgreSQL	<code>db.t3.micro</code>	auth, flag e targeting services
<code>elasticache/</code>	1 cluster Redis	<code>cache.t3.micro</code>	evaluation-service (cache em tempo real)
<code>dynamodb/</code>	Tabela ToggleMasterAnalytics	On-demand	analytics-service (eventos de flags)
<code>sqs/</code>	Fila togglemaster-events	Standard	Canal assíncrono evaluation → analytics

Todas as instâncias RDS foram provisionadas nas subnets privadas com security groups que permitem acesso apenas a partir dos CIDRs internos da VPC - nenhuma instância de banco possui acesso público.

Decisão: DynamoDB para analytics. O volume de eventos de avaliação de feature flags pode ser elevado e imprevisível. Escolhemos DynamoDB (NoSQL com billing por demanda)

para o analytics-service por sua capacidade de escalar automaticamente sem necessidade de pré-provisionar capacidade, o que seria o risco com uma instância RDS de tamanho fixo.

Decisão: SQS entre evaluation e analytics. O desacoplamento via fila garante que picos de avaliação de flags não derrubem o analytics-service. O evaluation-service publica eventos na fila e continua operando mesmo que o analytics-service esteja lento ou indisponível momentaneamente.

4.6 ECR - `modules/ecr/`

Provisionamos 5 repositórios privados no Amazon ECR, um por microserviço, usando `for_each` para evitar repetição de código:

```
auth-service | flag-service | targeting-service | evaluation-service | analytics-service
```

Justificativa para ECR em vez de Docker Hub: o ECR é privado (apenas a conta AWS acessa), reside na mesma rede da AWS (downloads sem custo de transferência entre ECR e EKS), e integra-se nativamente com IAM - a policy `AmazonEC2ContainerRegistryReadOnly` nos nodes é suficiente para autorizar o pull das imagens.

4.7 Backend Remoto S3 - `infra/backend.tf`

```
terraform {
  backend "s3" {
    bucket      = "togglemaster-tfstate-yuri-1"
    key         = "eks-cluster/terraform.tfstate"
    region      = "us-east-1"
    use_lockfile = true
  }
}
```

Justificativa para backend remoto: o `terraform.tfstate` mapeia todos os recursos gerenciados. Mantê-lo localmente em um projeto de equipe cria três problemas que precisávamos eliminar:

1. **Colaboração:** dois integrantes com estados diferentes tentariam gerenciar os mesmos recursos de formas divergentes, corrompendo a infraestrutura.
2. **Durabilidade:** a perda da máquina de qualquer integrante resultaria na perda do estado, tornando o Terraform incapaz de reconhecer os recursos existentes na AWS.
3. **Segurança:** o state pode conter valores sensíveis como endpoints de banco de dados. No bucket S3, o acesso é controlado via IAM.

`use_lockfile = true`: recurso do Terraform 1.10+ que grava um arquivo `.tflock` no bucket durante a execução de `terraform apply`. Se um segundo integrante iniciar um `apply` concorrente, o Terraform lê o `.tflock`, interrompe imediatamente com erro "state is locked" e informa qual operação detém o lock - evitando corrupção de estado por execuções paralelas.

4.8 Adaptação IAM para AWS Academy

O AWS Academy bloqueia as operações `iam:CreateRole`, `iam:CreatePolicy` e `iam:CreateOpenIDConnectProvider`. Adotamos a seguinte estratégia em todos os arquivos `iam.tf`:

Manter o código de conta pessoal comentado, com a alternativa para Academy ativa abaixo.

Exemplo - `infra/modules/cluster/iam.tf`:

```
# =====
# CONTA PESSOAL - código original (comentado para referência)
# =====
# resource "aws_iam_role" "eks_cluster_role" {
#   name = "${var.prefix}-eks-cluster-role"
#   assume_role_policy = jsonencode({
#     Version = "2012-10-17"
#     Statement = [{
#       Action    = "sts:AssumeRole"
#       Effect    = "Allow"
#       Principal = { Service = "eks.amazonaws.com" }
#     }]
#   })
# }
# resource "aws_iam_role_policy_attachment" "eks_cluster_AmazonEKSClusterPolicy" {
#   role            = aws_iam_role.eks_cluster_role.name
#   policy_arn      = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
# }

# =====
# AWS ACADEMY - referencia a LabRole pré-existente
# =====
data "aws_iam_role" "lab_role" {
  name = "LabRole"
}
```

Aplicamos o mesmo padrão em `managed-node-group/iam.tf` (role dos nodes EC2, com as três policy attachments: `AmazonEKSWorkerNodePolicy`, `AmazonEC2ContainerRegistryReadOnly`, `AmazonEKS_CNI_Policy`) e em `aws-loadbalancer-controller/iam.tf` (role IRSA com trust policy OIDC federada e `Condition` restringindo ao ServiceAccount `aws-load-balancer-controller` no namespace `kube-system`).

Justificativa: a abordagem com comentários demonstra o domínio completo da configuração de IAM para EKS - incluindo trust policies, policy attachments e IRSA com OIDC - mesmo sem poder executar o código no ambiente Academy.

4.9 Addons do Cluster - `modules/addons-eks/`

Todos os componentes do cluster foram instalados via Helm pelo Terraform, garantindo que a reinstalação do cluster seja completamente automatizada:

Componente	Chart Helm	Finalidade
ArgoCD	<code>argo-cd</code> v7.3.3	Operador GitOps
NGINX Ingress Controller	<code>ingress-nginx</code>	Roteamento de tráfego externo HTTP/HTTPS para os serviços

Componente	Chart Helm	Finalidade
ExternalDNS	<code>external-dns</code>	Criação automática de registros DNS no Route53 ao expor serviços
External Secrets Operator	<code>external-secrets</code>	Sincronização de segredos do AWS Secrets Manager para Secrets Kubernetes
KEDA	<code>keda</code>	Autoscaling orientado a eventos (ex: escalar pods baseado no tamanho da fila SQS)

Justificativa para instalação via Terraform: se instalados manualmente via `helm install`, esses componentes não seriam recriados em caso de destruição e recriação do cluster. Com `helm_release` no Terraform, um único `terraform apply` reconstrói o cluster completo com todos os addons configurados.

4.9 Execução do Terraform - Passo a Passo

A ordem de execução correta para provisionar o ambiente completo é a seguinte:

Pré-requisito: exportar credenciais da sessão AWS Academy

As credenciais temporárias do AWS Academy expiram a cada sessão. Antes de qualquer comando Terraform, é necessário exportá-las no terminal:

```
export AWS_ACCESS_KEY_ID="ASIA..."
export AWS_SECRET_ACCESS_KEY="..."
export AWS_SESSION_TOKEN="..."
export AWS_DEFAULT_REGION="us-east-1"

# Confirmar autenticação
aws sts get-caller-identity
```

Passo 1 - Criar o bucket S3 para o state (somente na primeira vez)

O bucket precisa existir antes do `terraform init`, pois o Terraform o utiliza para armazenar o state. É o único recurso criado manualmente fora do Terraform:

```
aws s3 mb s3://togglemaster-tfstate-yuri-1 --region us-east-1
```

Passo 2 - Inicializar o Terraform

```
cd infra/
terraform init
```

O `terraform init` baixa os providers declarados em `provider.tf` (AWS, Kubernetes, Helm, ArgoCD) e conecta ao backend S3 configurado em `backend.tf`. A partir deste ponto, todo estado é gerenciado remotamente.

Passo 3 - Visualizar o plano de execução

```
terraform plan
```

O `terraform plan` compara o estado atual (vazio na primeira execução) com os recursos declarados nos arquivos `.tf` e lista tudo que será criado, modificado ou destruído. Nenhuma alteração é feita na AWS neste passo - é exclusivamente uma simulação de leitura segura.

Passo 4 - Aplicar a infraestrutura

```
terraform apply -auto-approve
```

O `terraform apply` executa o plano e cria os recursos na AWS na ordem correta, respeitando as dependências entre módulos: rede → cluster EKS → node groups → bancos de dados → addons (ArgoCD, ExternalDNS, ExternalSecrets, KEDA) → aplicações ArgoCD (módulo `apps/`, que registra cada microserviço como `ArgoCD Application` apontando para as pastas `k8s/` do repositório). O state é gravado no S3 incrementalmente - se a execução for interrompida, pode ser retomada sem recriar recursos já existentes.

O provisionamento completo leva entre 15 e 25 minutos.

Passo 5 - Verificar recursos criados

```
# Cluster EKS
aws eks list-clusters --region us-east-1

# Bucket S3 com o state
aws s3 ls | grep togglemaster

# Bancos de dados RDS
aws rds describe-db-instances \
  --query 'DBInstances[*].DBInstanceIdentifier' \
  --output table
```

Destruição do ambiente (ao final)

```
terraform destroy -auto-approve
```

O `terraform destroy` remove todos os recursos provisionados na ordem inversa da criação. É fundamental executar ao final de cada sessão para não consumir créditos desnecessariamente.

5. Pipeline de CI/CD com DevSecOps - GitHub Actions

5.1 Arquitetura - Workflows Reutilizáveis

Criamos 7 arquivos de workflow em `.github/workflows/`:

Arquivo	Tipo	Função
<code>_ci-go.yml</code>	Reutilizável (<code>workflow_call</code>)	Pipeline base para serviços Go
<code>_ci-python.yml</code>	Reutilizável (<code>workflow_call</code>)	Pipeline base para serviços Python

Arquivo	Tipo	Função
auth-service.yml	Trigger	Chama _ci-go.yml com service: auth-service
evaluation-service.yml	Trigger	Chama _ci-go.yml com service: evaluation-service
flag-service.yml	Trigger	Chama _ci-python.yml com service: flag-service
targeting-service.yml	Trigger	Chama _ci-python.yml com service: targeting-service
analytics-service.yml	Trigger	Chama _ci-python.yml com service: analytics-service

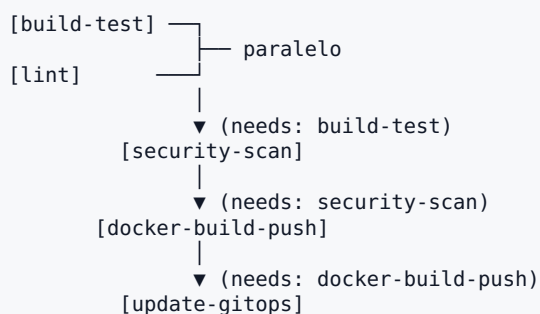
Justificativa para workflows reutilizáveis: sem o padrão `workflow_call`, cada serviço teria um arquivo de ~150 linhas repetido 5 vezes. Com workflows reutilizáveis, a lógica reside em dois arquivos e os 5 triggers são simples chamadas com parâmetros. Ao atualizar a versão do Trivy, por exemplo, basta alterar uma linha em `_ci-go.yml` e todos os 3 serviços Go herdam a mudança.

Filtro por path: cada trigger ativa o pipeline somente quando arquivos do seu serviço mudam:

```
on:
  push:
    branches: [main]
    paths: ["auth-service/**"]
```

Isso evita que a alteração de um serviço dispare desnecessariamente os pipelines de outros serviços - reduzindo consumo de minutos do GitHub Actions e tempo de feedback.

5.2 Fluxo dos Jobs



Jobs em paralelo: `build-test` e `lint` não têm dependência entre si e executam simultaneamente em máquinas distintas provisionadas pelo GitHub. Isso reduz o tempo de feedback para o desenvolvedor.

Bloqueio em cascata: qualquer job com `needs:` só executa se o job anterior concluiu com sucesso. A falha se propaga: se `security-scan` falhar, `docker-build-push` e `update-gitops` são

cancelados automaticamente. O código problemático não gera imagem, não chega ao ECR e não é deployado.

5.3 Job 1 - Build & Unit Test

Ferramenta	Go	Python
Compilação	<code>go build ./...</code>	N/A (interpretado)
Testes	<code>go test ./... -v -count=1</code>	<code>pytest</code>

-count=1 : força a reexecução dos testes sem usar o cache de resultados - garante que o resultado seja sempre da execução atual, não de um run anterior.

Justificativa da etapa: o princípio "fail fast" determina detectar falhas óbvias - erros de compilação e falhas de teste - antes de gastar tempo e recursos com ferramentas mais lentas como scanners de segurança.

5.4 Job 2 - Lint / Static Analysis

Ferramenta	Lingua gem	O que verifica
<code>golangci-lint</code> v1.59	Go	Agrega 50+ linters: variáveis não utilizadas, código morto, complexidade ciclomática, imports desnecessários
<code>flake8</code>	Python	PEP 8, erros de estilo, imports não usados

Versão fixada (v1.59): se não fixarmos a versão, uma atualização automática do linter pode introduzir novas regras que fazem o pipeline falhar sem mudança de código - o que gera confusão em revisão de PR. Versão fixada garante comportamento previsível.

5.5 Job 3 - Security Scan (DevSecOps)

Este é o job central da proposta DevSecOps. Implementamos dois tipos de análise de segurança complementares:

SCA - Software Composition Analysis

```
- name: Trivy SCA (filesystem)
  uses: aquasecurity/trivy-action@master
  with:
    scan-type: fs
    scan-ref: ${ inputs.service }
    severity: CRITICAL
    exit-code: "1"
    format: table
```

O Trivy varre o `go.sum` (dependências Go) ou `requirements.txt` (dependências Python) e cruza cada biblioteca com bases de dados de vulnerabilidades conhecidas: CVE (Common Vulnerabilities and Exposures), GitHub Advisory Database e OSV. Se uma dependência

possuir uma vulnerabilidade de severidade CRÍTICA com correção disponível, o scan retorna `exit-code: 1`.

As dependências Python dos serviços foram mantidas nas versões mais recentes (Flask 3.1, Werkzeug 3.1, requests 2.32, boto3 1.38, gunicorn 23) para minimizar a superfície de CVEs conhecidos nas dependências diretas.

SAST - Static Application Security Testing

```
# Go
- name: gosec
  uses: securego/gosec@master
  with:
    args: -severity high -confidence high -quiet ./${{ inputs.service }}/...

# Python
- name: bandit
  run: bandit -r ${{ inputs.service }} -ll
```

O `gosec` e o `bandit` analisam o código fonte sem executá-lo, identificando padrões inseguros: concatenação de strings em queries SQL (SQL injection), uso de funções de hash MD5/SHA1 para senhas, chamadas de sistema inseguras, entre outros. O filtro `-severity high -confidence high` reduz falsos positivos - reporta somente achados de alta severidade com alta confiança.

Regra de bloqueio - requisito do enunciado

```
severity: CRITICAL
exit-code: "1"
```

O `exit-code: "1"` faz o step retornar código de falha ao encontrar vulnerabilidade crítica. O GitHub Actions interpreta isso como falha do job, e o bloqueio em cascata cancela `docker-build-push` e `update-gitops`. **O código vulnerável jamais gera imagem Docker e jamais é deployado no cluster.**

5.6 Job 4 - Docker Build, Scan & Push to ECR

Rastreabilidade com hash do commit

```
- name: Set image tag
  id: set-tag
  run: echo "tag=v1.0.0-$(echo $GITHUB_SHA | head -c 7)" >> "$GITHUB_OUTPUT"
```

A tag `v1.0.0-<7-chars-SHA>` cria rastreabilidade bidirecional: dado um pod rodando em produção, é possível identificar o commit exato que o originou. Dado um commit no GitHub, identifica-se a imagem gerada.

O valor é exportado via `$GITHUB_OUTPUT` e consumido pelo job seguinte como `${{ needs.docker-build-push.outputs.image_tag }}` - o GitHub Actions resolve a referência automaticamente sem necessidade de variáveis manuais entre jobs.

Scan da imagem Docker (segundo scan Trivy)

```
- name: Trivy container scan
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ inputs.ecr_repo }:${ steps.set-tag.outputs.tag }
    severity: CRITICAL
    exit-code: "1"
    ignore-unfixed: true
```

O scan do filesystem (job 3) verifica as dependências declaradas no código. Porém, a imagem Docker também contém um sistema operacional base - Alpine Linux ou Debian - com bibliotecas próprias. Uma vulnerabilidade no `openssl` do Alpine, por exemplo, não aparece no `go.sum`. O segundo scan, na imagem construída, cobre essa camada adicional. Os dois scans se complementam.

ignore-unfixed: true: o scan falha somente em CVEs que já possuem correção disponível. CVEs sem patch publicado pelo mantenedor do pacote ou da distribuição são reportados mas não bloqueiam o pipeline. Isso evita falsos bloqueios causados por vulnerabilidades no OS base para as quais não há ação possível do time de desenvolvimento.

Autenticação AWS no pipeline

```
- name: Configurar credenciais AWS
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
    aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    aws-session-token: ${ secrets.AWS_SESSION_TOKEN }
    aws-region: us-east-1
```

A abordagem ideal para autenticação entre GitHub Actions e AWS é OIDC (OpenID Connect): o GitHub emite um token criptográfico assinado que a AWS valida via um OIDC Provider configurado na IAM, sem que nenhuma credencial estática seja armazenada. O código original para essa configuração (`role-to-assume: LabRole`) está preservado nos comentários dos arquivos `iam.tf`.

Adaptação para AWS Academy: o Academy bloqueia `iam:CreateOpenIDConnectProvider`, impossibilitando a criação do OIDC Provider. Adotamos portanto credenciais estáticas armazenadas como secrets no repositório GitHub (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`). As credenciais são renovadas manualmente a cada sessão Academy antes de acionar o pipeline - limitação conhecida e documentada como adaptação do ambiente.

5.7 Job 5 - Update GitOps Manifest

```
- name: Atualizar tag no deployment.yaml
  run: |
    IMAGE_URI=${{ steps.login-ecr.outputs.registry }}/${{ inputs.ecr_repo }}
    NEW_TAG=${{ needs.docker-build-push.outputs.image_tag }}
    sed -i "s|image: ${IMAGE_URI}.*|image: ${IMAGE_URI}:${NEW_TAG}|g" \
      ${inputs.service }}/k8s/deployment.yaml

- name: Commit e push do manifesto atualizado
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    git add ${inputs.service }}/k8s/deployment.yaml
    git diff --cached --quiet || git commit -m "ci: atualiza imagem ${inputs.service} para $
      {{ needs.docker-build-push.outputs.image_tag }}"
    git push
```

Este job fecha o ciclo CI → CD. Após a imagem ser publicada no ECR, o pipeline edita o `deployment.yaml` do serviço - substituindo a tag anterior pela nova - e realiza commit e push no repositório com o autor `github-actions[bot]`.

`git config user.name "github-actions[bot]"`: identifica o commit como automático no `git log`, permitindo distinguir deploys automatizados de commits humanos - relevante para auditoria do histórico.

`git diff --staged --quiet || git commit`: verifica se há alteração real antes de commitar. Se a imagem não mudou (por exemplo, um push que não altera o serviço), o `git diff` não encontra mudança e o commit é pulado - evitando commits vazios que poluiriam o histórico.

Este commit é o gatilho para o ArgoCD detectar a mudança e iniciar o deploy.

5.8 Resumo das Ferramentas DevSecOps

Ferramenta	Job	Tipo	Language m	O que detecta
<code>golangci-lint</code>	lint	Linter	Go	Qualidade, padrões, código morto
<code>flake8</code>	lint	Linter	Python	PEP 8, sintaxe
Trivy (fs)	security-scan	SCA	Go/Python	CVEs em dependências
<code>gosec</code>	security-scan	SAST	Go	SQL injection, criptografia fraca
<code>bandit</code>	security-scan	SAST	Python	Injeções, módulos inseguros
Trivy (image)	docker-build-push	Container Scan	Docker	CVEs no OS base da imagem

6. Entrega Contínua e GitOps - ArgoCD

6.1 O Modelo GitOps

A abordagem tradicional de deploy (push model) usa scripts ou pipelines que executam `kubectl apply` diretamente no cluster. Isso gera os problemas do enunciado: deploys manuais, falta de rastreabilidade, risco de deriva de configuração.

GitOps inverte esse modelo para um **pull model**: o operador GitOps roda dentro do próprio cluster e monitora o repositório Git. Quando detecta divergência entre o estado declarado no repositório e o estado atual do cluster, ele reconcilia - aplicando a diferença.

Escolhemos este modelo pelas seguintes consequências operacionais:

Benefício	Como se manifesta
Auditoria completa	O histórico de deploys é o <code>git log</code> do repositório
Rollback trivial	<code>git revert</code> reverte o deploy; ArgoCD aplica em minutos
Deriva zero	Mudanças manuais via <code>kubectl</code> são revertidas pelo ArgoCD automaticamente
Segurança	Nenhuma máquina externa precisa de credenciais para acessar o cluster diretamente

6.2 Estrutura dos Manifestos Kubernetes

Cada microsserviço possui uma pasta `k8s/` com os manifestos YAML que descrevem seu estado desejado no cluster:

```
auth-service/k8s/
├── namespace.yaml      ← namespace isolado para o serviço
├── deployment.yaml     ← definição dos pods, réplicas, imagem, recursos
├── service.yaml        ← endereço de rede estável para os pods
├── configmap.yaml      ← variáveis de configuração não-sensíveis
├── external-secret.yaml ← busca senhas do AWS Secrets Manager via ESO
└── hpa.yaml            ← autoscaling horizontal de pods
```

Por que um namespace por serviço? O isolamento por namespace permite aplicar políticas de rede e de acesso específicas por serviço, facilita a visualização no ArgoCD e evita colisões de nome entre recursos.

Relação entre os manifestos e o ArgoCD: o ArgoCD monitora o repositório `github.com/yuriycarmo/tech-challenge-3` e observa as pastas `k8s/` de cada serviço. Quando detecta que um arquivo YAML mudou - como a tag da imagem no `deployment.yaml` - compara o estado declarado com o estado do cluster e aplica a diferença via `kubectl apply` internamente.

6.3 O `deployment.yaml` - Ponto de Integração CI/CD ↔ GitOps

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: auth
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: auth-service
          image: <ACCOUNT>.dkr.ecr.us-east-1.amazonaws.com/auth-service:v1.0.0-abc1234
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "200m"
          envFrom:
            - secretRef:
                name: auth-service-secret
            - configMapRef:
                name: auth-service-config
```

A linha `image:` é o ponto de integração entre o pipeline CI/CD e o GitOps. O job `update-gitops` substitui essa tag a cada build bem-sucedido. O ArgoCD detecta a mudança no repositório e inicia um **rolling update** - substituindo os pods um a um sem interromper o serviço (zero downtime).

`resources.requests` e `resources.limits`: o Kubernetes usa `requests` para alocar o pod em um node com recursos disponíveis, e `limits` para evitar que um pod monopolize o node. A definição explícita desses valores é prática obrigatória em produção - sem ela, o Kubernetes não consegue fazer scheduling eficiente.

`secretRef: auth-service-secret`: as senhas (banco de dados, chaves de API) não são armazenadas no repositório. O External Secrets Operator busca os valores no AWS Secrets Manager e cria um Secret nativo do Kubernetes. O pod carrega esses valores como variáveis de ambiente em tempo de inicialização.

Justificativa para não buscar secrets diretamente da AWS: os pods do Kubernetes não têm configuração de credenciais AWS por padrão - o ESO já tem as permissões necessárias e atua como intermediário. Centralizar a busca no ESO simplifica a configuração dos serviços e elimina a necessidade de configurar IAM por pod (exceto nos casos de IRSA, que não está disponível no Academy).

6.4 Instalação do ArgoCD via Terraform

```
resource "helm_release" "argocd" {
  name           = "argocd"
  repository     = "https://argoproj.github.io/argo-helm"
  chart          = "argo-cd"
  namespace     = "argocd"
  create_namespace = true
  version       = "7.3.3"

  set {
    name = "server.ingress.enabled"
    value = "true"
  }
}
```

Por que instalar via Terraform? Para que a instalação do ArgoCD seja ela mesma infraestrutura como código. Se o cluster for destruído e recriado, o `terraform apply` reinstala o ArgoCD automaticamente, sem intervenção manual da equipe.

version = "7.3.3": versão fixada para garantir reprodutibilidade - qualquer rebuild do ambiente instalará exatamente a mesma versão, evitando diferenças de comportamento por atualizações silenciosas.

6.5 Configuração de Sync Automático

```
sync_policy {
  automated {
    prune      = true
    self_heal = true
  }
}
```

- **prune = true**: se um recurso for removido do repositório Git (ex: deletar um `configmap.yaml`), o ArgoCD o remove também do cluster. Sem essa opção, recursos obsoletos acumulariam indefinidamente.
- **self_heal = true**: se um operador executar `kubectl edit` ou `kubectl delete` diretamente no cluster, o ArgoCD detecta a divergência e restaura o estado do repositório em até 3 minutos. O Git é sempre a fonte de verdade.

7. Decisões Técnicas Justificadas

7.1 Infraestrutura Terraform como Código (não scripts)

Optamos por Terraform em vez de scripts AWS CLI ou CloudFormation pelos seguintes motivos:

- **Idempotência**: `terraform apply` pode ser executado múltiplas vezes com o mesmo resultado - apenas mudanças reais são aplicadas, recursos já existentes não são recriados.
- **Plan antes de Apply**: `terraform plan` mostra exatamente o que será criado, modificado ou destruído antes de executar. Isso permite revisão antes de alterar infraestrutura real.

- **State centralizado:** o `tfstate` no S3 com lockfile garante que a equipe trabalhe com a mesma visão da infraestrutura.
- **Provider AWS maduro:** o provider Terraform para AWS cobre virtualmente todos os serviços - EKS, RDS, ElastiCache, DynamoDB, SQS, ECR, IAM - com recursos bem documentados.

7.2 Workflows Reutilizáveis em vez de Duplicação

Centralizar a lógica em `_ci-go.yml` e `_ci-python.yml` e chamar esses workflows nos 5 triggers segue o princípio DRY (Don't Repeat Yourself). O custo de manutenção cai: uma melhoria de segurança no pipeline beneficia todos os serviços automaticamente.

7.3 Dois Scans Trivy (filesystem + image)

Implementamos duas varreduras Trivy distintas porque elas cobrem ameaças complementares:

- **Scan filesystem:** analisa as dependências declaradas (`go.sum` , `requirements.txt`) antes de construir a imagem. Falha rápido, antes de gastar tempo construindo.
- **Scan de imagem:** analisa a imagem construída, incluindo o OS base (Alpine, Debian). Vulnerabilidades no sistema operacional base não aparecem no scan de filesystem.

7.4 Manter Código Original IAM em Comentários

Em vez de simplesmente substituir o código de conta pessoal pelo `data source` do Academy, preservamos o código original integralmente nos comentários. Essa decisão serve dois propósitos:

1. **Demonstração de conhecimento:** o código comentado evidencia o domínio completo de IAM para EKS - criação de roles, trust policies, policy attachments e IRSA - mesmo sem poder executar no ambiente Academy.
2. **Referência futura:** o mesmo repositório pode ser usado como base para uma implementação em conta pessoal, com as roles sendo ativadas simplesmente descomentando o código.

8. Segurança do Repositório

Implementamos medidas para que o repositório pudesse ser público no GitHub sem expor informações sensíveis:

Medida	Implementação	Risco mitigado
<code>terraform.tfstate</code> fora do repo	<code>.gitignore</code> com <code>*.tfstate</code>	Exposição de configurações internas da infraestrutura
<code>*.tfvars</code> fora do repo	<code>.gitignore</code> com <code>*.tfvars</code>	Tokens e chaves em texto plano
Token ArgoCD via env	<code>TF_VAR_argocd_auth_token</code> em vez de arquivo	Token JWT em repositório público

Medida	Implementação	Risco mitigado
Template de variáveis	<code>infra/terraform.tfvars.example</code> com placeholders	Onboarding sem arquivos sensíveis
Senhas docker-compose via env	<code>\${POSTGRES_AUTH_PASSWORD:-valor_local}</code>	Senhas hardcoded em arquivo commitado
<code>.env.example</code>	Template com instruções	Facilita configuração local sem expor credenciais reais
Secrets GitHub para pipeline	<code>AWS_ACCESS_KEY_ID/SECRET/SESSION_TOKEN</code> como repository secrets	Credenciais hardcoded em arquivos de workflow
Secrets via ESO	Nenhuma senha nos manifests YAML	Credenciais em texto nos arquivos Kubernetes

9. Desafios e Limitações

9.1 Restrições do AWS Academy

O principal desafio técnico foi operar dentro das restrições do ambiente Academy. A documentação oficial da AWS e a maioria dos tutoriais assumem conta pessoal com permissões completas de IAM. Identificamos as operações bloqueadas iterativamente - `iam:CreateRole`, `iam:CreatePolicy`, `iam:CreateOpenIDConnectProvider` - e desenvolvemos a estratégia de `data source` + comentários preservando o código original.

9.2 Remoção do `tfstate` do Histórico Git

O arquivo `terraform.tfstate` havia sido commitado antes da configuração do backend remoto. Removemos o arquivo do tracking com `git rm --cached terraform.tfstate`, mantendo-o localmente. O `.gitignore` garante que não seja reintroduzido. O arquivo de state continha endpoints de banco de dados e outros detalhes da infraestrutura - o risco de mantê-lo no histórico público foi identificado e endereçado.

9.3 IRSA Indisponível no Academy

Sem o OIDC Provider no IAM, não foi possível implementar IRSA - a abordagem recomendada para permissões granulares por pod. No ambiente Academy, todos os pods herdam as permissões amplas da `LabRole` via node EC2. Em produção real, adotaríamos IRSA obrigatoriamente para isolar as permissões de cada serviço: o `analytics-service` acessaria apenas DynamoDB e SQS, o `auth-service` acessaria apenas seu RDS, e assim por diante. O código para essa implementação está preservado nos comentários dos arquivos `iam.tf`.

9.4 Credenciais Academy e o pipeline CI/CD

As credenciais do Academy expiram a cada sessão (geralmente 4 horas). O Academy bloqueia `iam:CreateOpenIDConnectProvider`, impedindo a configuração do OIDC Provider que permitiria autenticação sem credenciais estáticas. A solução adotada foi armazenar as três credenciais temporárias (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`) como

repository secrets no GitHub, renovando-as manualmente antes de cada execução do pipeline. Operações locais como `terraform apply` e `aws eks update-kubeconfig` exigem o mesmo procedimento de renovação.

10. Estimativa de Custos AWS

(Inserir print do AWS Cost Explorer ou AWS Pricing Calculator)

Estimativa mensal de referência para o ambiente em execução contínua (us-east-1):

Serviço	Configuração	Custo Estimado/mês
EKS Cluster	1 cluster	~\$73
EC2 Node Group	2× t3.medium	~\$60
RDS PostgreSQL	3× db.t3.micro	~\$45
ElastiCache Redis	1× cache.t3.micro	~\$12
NAT Gateway	2 instâncias	~\$65
Application Load Balancer	1 ALB	~\$16
ECR	5 repositórios, armazenamento mínimo	~\$1
S3	Backend tfstate	< \$1
Route53	1 hosted zone	~\$0,50
Total estimado		~\$273/mês

Os valores acima são estimativas baseadas em preços da região us-east-1 com uso contínuo 24/7. Para o contexto do AWS Academy Learner Lab, os créditos disponíveis cobrem esse custo por período limitado. Em produção real, recomendamos avaliar Savings Plans para os EC2 (redução de ~30%) e instâncias Spot para os nodes de workloads tolerantes a interrupção.

Pós-Graduação em Software Engineering - POSTECH - 2026